

Wide Residual Networks

Sergey Zagoruyko
 sergey.zagoruyko@enpc.fr
 Nikos Komodakis
 nikos.komodakis@enpc.fr

Université Paris-Est, École des Ponts
 ParisTech
 Paris, France

Abstract

Deep residual networks were shown to be able to scale up to thousands of layers and still have improving performance. However, each fraction of a percent of improved accuracy costs nearly doubling the number of layers, and so training very deep residual networks has a problem of diminishing feature reuse, which makes these networks very slow to train. To tackle these problems, in this paper we conduct a detailed experimental study on the architecture of ResNet blocks, based on which we propose a novel architecture where we decrease depth and increase width of residual networks. We call the resulting network structures wide residual networks (WRNs) and show that these are far superior over their commonly used thin and very deep counterparts. For example, we demonstrate that even a simple 16-layer-deep wide residual network outperforms in accuracy and efficiency all previous deep residual networks, including thousand-layer-deep networks, achieving new state-of-the-art results on CIFAR, SVHN, COCO, and significant improvements on ImageNet. Our code and models are available at <https://github.com/szagoruyko/wide-residual-networks>.

Introduction

Convolutional neural networks have seen a gradual increase of the number of layers in the last few years, starting from AlexNet [16], VGG [26], Inception [60] to Residual [10] networks, corresponding to improvements in many image recognition tasks. The superiority of deep networks has been spotted in several works in the recent years [8, 22]. However, training deep neural networks has several difficulties, including exploding/vanishing gradients and degradation. Various techniques were suggested to enable training of deeper neural networks, such as well-designed initialization strategies [11, 12], better optimizers [29], skip connections [19, 23], knowledge transfer [4, 24] and layer-wise training [25].

The latest residual networks [10] had a large success winning ImageNet and COCO 2015 competition and achieving state-of-the-art in several benchmarks, including object classification on ImageNet and CIFAR, object detection and segmentation on PASCAL VOC and MS COCO. Compared to Inception architectures they show better generalization, meaning the features can be utilized in transfer learning with better efficiency. Also, follow-up work showed that residual links speed up convergence of deep networks [61]. Recent follow-up work explored the order of activations in residual networks, presenting identity mappings in residual blocks [13] and improving training of very deep networks. Successful training of very deep networks was also shown to be possible through the use of highway networks

(a) basic (b) bottleneck (c) basic-wide (d) wide-dropout

Figure 1: Various residual blocks used in the paper. Batch normalization and ReLU precede each convolution (omitted for clarity)

[18], which is an architecture that had been proposed prior to residual networks. The essential difference between residual and highway networks is that in the latter residual links are gated and weights of these gates are learned.

Therefore, up to this point, the study of residual networks has focused mainly on the order of activations inside a ResNet block and the depth of residual networks. In this work we attempt to conduct an experimental study that goes beyond the above points. By doing so, our goal is to explore a much richer set of network architectures of ResNet blocks and thoroughly examine how several other different aspects besides the order of activations affect performance. As we explain below, such an exploration of architectures has led to new interesting findings with great practical importance concerning residual networks.

Width vs depth in residual networks. The problem of shallow vs deep networks has been in discussion for a long time in machine learning [9, 18] with pointers to the circuit complexity theory literature showing that shallow circuits can require exponentially more components than deeper circuits. The authors of residual networks tried to make them as thin as possible in favor of increasing their depth and having less parameters, and even introduced a «bottleneck» block which makes ResNet blocks even thinner.

We note, however, that the residual block with identity mapping that allows to train very deep networks is at the same time a weakness of residual networks. As gradient flows through the network there is nothing to force it to go through residual block weights and it can avoid learning anything during training, so it is possible that there is either only a few blocks that learn useful representations, or many blocks share very little information with small contribution to the final goal. This problem was formulated as diminishing feature reuse in [28]. The authors of [14] tried to address this problem with the idea of randomly disabling residual blocks during training. This method can be viewed as a special case of dropout [24], where each residual block has an identity scalar weight on which dropout is applied. The effectiveness of this approach proves the hypothesis above.

Motivated by the above observation, our work builds on top of [13] and tries to answer the question of how wide deep residual networks should be and address the problem of training. In this context, we show that the widening of ResNet blocks (if done properly) provides a much more effective way of improving performance of residual networks compared to increasing their depth. In particular, we present wider deep residual networks that significantly improve over [13], having 50 times less layers and being more than 2 times faster. We call the resulting network architectures *wide residual networks*. For instance, our wide 16-layer deep network has the same accuracy as a 1000-layer thin deep network and a comparable number of parameters, although being several times faster to train. This type of experiments

thus seem to indicate that the main power of deep residual networks is in residual blocks, and that the effect of depth is supplementary. We note that one can train even better wide residual networks that have twice as many parameters (and more), which suggests that to further improve performance by increasing depth of thin networks one needs to add thousands of layers in this case.

Use of dropout in ResNet blocks. Dropout was first introduced in [27] and then was adopted by many successful architectures as [46, 28] etc. It was mostly applied on top layers that had a large number of parameters to prevent feature coadaptation and overfitting. It was then mainly substituted by batch normalization [15] which was introduced as a technique to reduce internal covariate shift in neural network activations by normalizing them to have specific distribution. It also works as a regularizer and the authors experimentally showed that a network with batch normalization achieves better accuracy than a network with dropout. In our case, as widening of residual blocks results in an increase of the number of parameters, we studied the effect of dropout to regularize training and prevent overfitting. Previously, dropout in residual networks was studied in [43] with dropout being inserted in the identity part of the block, and the authors showed negative effects of that. Instead, we argue here that dropout should be inserted between convolutional layers. Experimental results on wide residual networks show that this leads to consistent gains, yielding even new state-of-the-art results (e.g., 16-layer-deep wide residual network with dropout achieves 1.64% error on SVHN).

In summary, the contributions of this work are as follows:

- We present a detailed experimental study of residual network architectures that thoroughly examines several important aspects of ResNet block structure.
- We propose a novel *widened* architecture for ResNet blocks that allows for residual networks with significantly improved performance.
- We propose a new way of utilizing dropout within deep residual networks so as to properly regularize them and prevent overfitting during training.
- Last, we show that our proposed ResNet architectures achieve state-of-the-art results on several datasets dramatically improving accuracy and speed of residual networks.

2 Wide residual networks

Residual block with identity mapping can be represented by the following formula:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l) \quad (1)$$

where x_{l+1} and x_l are input and output of the l -th unit in the network, $\mathcal{F}$ is a residual function and $\mathcal{W}_l$ are parameters of the block. Residual network consists of sequentially stacked residual blocks.

In [43] residual networks consisted of two type of blocks:

- *basic* - with two consecutive 3×3 convolutions with batch normalization and ReLU preceding convolution: $\text{conv}3 \times 3 - \text{conv}3 \times 3$ Fig.1(a)
- *bottleneck* - with one 3×3 convolution surrounded by dimensionality reducing and expanding 1×1 convolution layers: $\text{conv}1 \times 1 - \text{conv}3 \times 3 - \text{conv}1 \times 1$ Fig.1(b)

group name	output size	block type = $B(3, 3)$
conv1	32×32	$[3 \times 3, 16]$
conv2	32×32	$\begin{bmatrix} 3 \times 3, 16 \times k \\ 3 \times 3, 16 \times k \end{bmatrix} \times N$
conv3	16×16	$\begin{bmatrix} 3 \times 3, 32 \times k \\ 3 \times 3, 32 \times k \end{bmatrix} \times N$
conv4	8×8	$\begin{bmatrix} 3 \times 3, 64 \times k \\ 3 \times 3, 64 \times k \end{bmatrix} \times N$
avg-pool	1×1	$[8 \times 8]$

Table 1: Structure of wide residual networks. Network width is determined by factor k . Original architecture [13] is equivalent to $k = 1$. Groups of convolutions are shown in brackets where N is a number of blocks in group, downsampling performed by the first layers in groups conv3 and conv4. Final classification layer is omitted for clearance. In the particular example shown, the network uses a ResNet block of type $B(3, 3)$.

Compared to the original architecture [13] in [26] the order of batch normalization, activation and convolution in residual block was changed from conv-BN-ReLU to BN-ReLU-conv. As the latter was shown to train faster and achieve better results we don't consider the original version. Furthermore, so-called «bottleneck» blocks were initially used to make blocks less computationally expensive to increase the number of layers. As we want to study the effect of widening and «bottleneck» is used to make networks thinner we don't consider it too, focusing instead on «basic» residual architecture.

There are essentially three simple ways to increase representational power of residual blocks:

- to add more convolutional layers per block
- to widen the convolutional layers by adding more feature planes
- to increase filter sizes in convolutional layers

As small filters were shown to be very effective in several works including [26, 31] we do not consider using filters larger than 3×3 . Let us also introduce two factors, deepening factor l and widening factor k , where l is the number of convolutions in a block and k multiplies the number of features in convolutional layers, thus the baseline «basic» block corresponds to $l = 2, k = 1$. Figures 1(a) and 1(c) show schematic examples of «basic» and «basic-wide» blocks respectively.

The general structure of our residual networks is illustrated in table 1: it consists of an initial convolutional layer conv1 that is followed by 3 groups (each of size N) of residual blocks conv2, conv3 and conv4, followed by average pooling and final classification layer. The size of conv1 is fixed in all of our experiments, while the introduced widening factor k scales the width of the residual blocks in the three groups conv2–4 (e.g., the original «basic» architecture is equivalent to $k = 1$). We want to study the effect of representational power of residual block and, to that end, we perform and test several modifications to the «basic» architecture, which are detailed in the following subsections.

2.1 Type of convolutions in residual block

Let $B(M)$ denote residual block structure, where M is a list with the kernel sizes of the convolutional layers in a block. For example, $B(3, 1)$ denotes a residual block with 3×3 and 1×1 convolutional layers (we always assume square spatial kernels). Note that, as we do not consider «bottleneck» blocks as explained earlier, the number of feature planes is always kept the same across the block. We would like to answer the question of how important each of the 3×3 convolutional layers of the «basic» residual architecture is and if they can be substituted by a less computationally expensive 1×1 layer or even a combination of 1×1 and 3×3 convolutional layers, *e.g.*, $B(1, 3)$ or $B(1, 3)$. This can increase or decrease the representational power of the block. We thus experiment with the following combinations (note that the last combination, *i.e.*, $B(3, 1, 1)$ is similar to effective Network-in-Network [20] architecture):

1. $B(3, 3)$ - original «basic» block
2. $B(3, 1, 3)$ - with one extra 1×1 layer
3. $B(1, 3, 1)$ - with the same dimensionality of all convolutions, «straightened» bottleneck
4. $B(1, 3)$ - the network has alternating 1×1 - 3×3 convolutions everywhere
5. $B(3, 1)$ - similar idea to the previous block
6. $B(3, 1, 1)$ - Network-in-Network style block

2.2 Number of convolutional layers per residual block

We also experiment with the block deepening factor l to see how it affects performance. The comparison has to be done among networks with the same number of parameters, so in this case we need to build networks with different l and d (where d denotes the total number of blocks) while ensuring that network complexity is kept roughly constant. This means, for instance, that d should decrease whenever l increases.

2.3 Width of residual blocks

In addition to the above modifications, we experiment with the widening factor k of a block. While the number of parameters increases linearly with l (the deepening factor) and d (the number of ResNet blocks), number of parameters and computational complexity are quadratic in k . However, it is more computationally effective to widen the layers than have thousands of small kernels as GPU is much more efficient in parallel computations on large tensors, so we are interested in an optimal d to k ratio.

One argument for wider residual networks would be that almost all architectures before residual networks, including the most successful Inception [60] and VGG [74], were much wider compared to [3]. For example, residual networks WRN-22-8 and WRN-16-10 (see next paragraph for explanation of this notation) are very similar in width, depth and number of parameters to VGG architectures.

We further refer to original residual networks with $k = 1$ as «thin» and to networks with $k > 1$ as «wide». In the rest of the paper we use the following notation: WRN- n - k denotes a residual network that has a total number of convolutional layers n and a widening factor k (for example, network with 40 layers and $k = 2$ times wider than original would be denoted as WRN-40-2). Also, when applicable we append block type, *e.g.* WRN-40-2- $B(3, 3)$.

block type	depth	# params	time,s	CIFAR-10
$B(1, 3, 1)$	40	1.4M	85.8	6.06
$B(3, 1)$	40	1.2M	67.5	5.78
$B(1, 3)$	40	1.3M	72.2	6.42
$B(3, 1, 1)$	40	1.3M	82.2	5.86
$B(3, 3)$	28	1.5M	67.5	5.73
$B(3, 1, 3)$	22	1.1M	59.9	5.78

Table 2: Test error (% , median over 5 runs) on CIFAR-10 of residual networks with $k = 2$ and different block types. Time column measures one training epoch.

l	CIFAR-10
1	6.69
2	5.43
3	5.65
4	5.93

Table 3: Test error (% , median over 5 runs) on CIFAR-10 of WRN-40-2 (2.2M) with various l .

2.4 Dropout in residual blocks

As widening increases the number of parameters we would like to study ways of regularization. Residual networks already have batch normalization that provides a regularization effect, however it requires heavy data augmentation, which we would like to avoid, and it's not always possible. We add a dropout layer into each residual block between convolutions as shown in fig. 1(d) and after ReLU to perturb batch normalization in the next residual block and prevent it from overfitting. In very deep residual networks that should help deal with diminishing feature reuse problem enforcing learning in different residual blocks.

3 Experimental results

For experiments we chose well-known CIFAR-10, CIFAR-100, SVHN and ImageNet image classification datasets. CIFAR-10 and CIFAR-100 datasets [14] consist of 32×32 color images drawn from 10 and 100 classes split into 50,000 train and 10,000 test images. For data augmentation we do horizontal flips and take random crops from image padded by 4 pixels on each side, filling missing pixels with reflections of original image. We don't use heavy data augmentation as proposed in [9]. SVHN is a dataset of Google's Street View House Numbers images and contains about 600,000 digit images, coming from a significantly harder real world problem. For experiments on SVHN we don't do any image preprocessing, except dividing images by 255 to provide them in $[0,1]$ range as input. All of our experiments except ImageNet are based on [13] architecture with pre-activation residual blocks and we use it as baseline. For ImageNet, we find that using pre-activation in networks with less than 100 layers does not make any significant difference and so we decide to use the original ResNet architecture in this case. Unless mentioned otherwise, for CIFAR we follow the image preprocessing of [9] with ZCA whitening. However, for some CIFAR experiments we instead use simple mean/std normalization such that we can directly compare with [13] and other ResNet related works that make use of this type of preprocessing.

In the following we describe our findings w.r.t. the different ResNet block architectures and also analyze the performance of our proposed wide residual networks. We note that for all experiments related to «type of convolutions in a block» and «number of convolutions per block» we use $k = 2$ and reduced depth compared to [13] in order to speed up training.

Type of convolutions in a block

We start by reporting results using trained networks with different block types B (reported results are on CIFAR-10). We used WRN-40-2 for blocks $B(1,3,1)$, $B(3,1)$, $B(1,3)$ and $B(3,1,1)$ as these blocks have only one 3×3 convolution. To keep the number of parameters comparable we trained other networks with less layers: WRN-28-2- $B(3,3)$ and WRN-22-2- $B(3,1,3)$. We provide the results including test accuracy in median over 5 runs and time per training epoch in the table 2. Block $B(3,3)$ turned out to be the best by a little margin, and $B(3,1)$ with $B(3,1,3)$ are very close to $B(3,3)$ in accuracy having less parameters and less layers. $B(3,1,3)$ is faster than others by a small margin.

Based on the above, blocks with comparable number of parameters turned out to give more or less the same results. Due to this fact, we hereafter restrict our attention to only WRNs with 3×3 convolutions so as to be also consistent with other methods.

Number of convolutions per block

We next proceed with the experiments related to varying the deepening factor l (which represents the number of convolutional layers per block). We show indicative results in table 3, where in this case we took WRN-40-2 with 3×3 convolutions and trained several networks with different deepening factor $l \in [1, 2, 3, 4]$, same number of parameters (2.2×10^6) and same number of convolutional layers.

As can be noticed, $B(3,3)$ turned out to be the best, whereas $B(3,3,3)$ and $B(3,3,3,3)$ had the worst performance. We speculate that this is probably due to the increased difficulty in optimization as a result of the decreased number of residual connections in the last two cases. Furthermore, $B(3)$ turned out to be quite worse. The conclusion is that $B(3,3)$ is optimal in terms of number of convolutions per block. For this reason, in the remaining experiments we only consider wide residual networks with a block of type $B(3,3)$.

Width of residual blocks

As we try to increase widening parameter k we have to decrease total number of layers. To find an optimal ratio we experimented with k from 2 to 12 and depth from 16 to 40. The results are presented in table 4. As can be seen, all networks with 40, 22 and 16 layers see consistent gains when width is increased by 1 to 12 times. On the other hand, when keeping the same fixed widening factor $k = 8$ or $k = 10$ and varying depth from 16 to 28 there is a consistent improvement, however when we further increase depth to 40 accuracy decreases (e.g., WRN-40-8 loses in accuracy to WRN-22-8).

We show additional results in table 5 where we compare thin and wide residual networks. As can be observed, wide WRN-40-4 compares favorably to thin ResNet-1001 as it achieves better accuracy on both CIFAR-10 and CIFAR-100. Yet, it is interesting that these networks have comparable number of parameters, 8.9×10^6 and 10.2×10^6 , suggesting that depth does not add regularization effects compared to width at this level. As we show further in benchmarks, WRN-40-4 is 8 times faster to train, so evidently depth to width ratio in the original thin residual networks is far from optimal.

Also, wide WRN-28-10 outperforms thin ResNet-1001 by 0.92% (with the same mini-batch size during training) on CIFAR-10 and 3.46% on CIFAR-100, having 36 times less layers (see table 5). We note that the result of 4.64% with ResNet-1001 was obtained with batch size 64, whereas we use a batch size 128 in all of our experiments (i.e., all other results

depth	k	# params	CIFAR-10	CIFAR-100
40	1	0.6M	6.85	30.89
40	2	2.2M	5.33	26.04
40	4	8.9M	4.97	22.89
40	8	35.7M	4.66	-
28	10	36.5M	4.17	20.50
28	12	52.5M	4.33	20.43
22	8	17.2M	4.38	21.22
22	10	26.8M	4.44	20.75
16	8	11.0M	4.81	22.07
16	10	17.1M	4.56	21.59

Table 4: Test error (%) of various wide networks on CIFAR-10 and CIFAR-100 (ZCA pre-processing).

reported in table 5 are with batch size 128). Training curves for these networks are presented in Figure 2.

Despite previous arguments that depth gives regularization effects and width causes network to overfit, we successfully train networks with several times more parameters than ResNet-1001. For instance, wide WRN-28-10 (table 5) and wide WRN-40-10 (table 9) have respectively 3.6 and 5 times more parameters than ResNet-1001 and both outperform it by a significant margin.

	depth- k	# params	CIFAR-10	CIFAR-100
NIN [20]			8.81	35.67
DSN [19]			8.22	34.57
FitNet [24]			8.39	35.04
Highway [28]			7.72	32.39
ELU [8]			6.55	24.28
original-ResNet[11]	110	1.7M	6.43	25.16
	1202	10.2M	7.93	27.82
stoc-depth[12]	110	1.7M	5.23	24.58
	1202	10.2M	4.91	-
pre-act-ResNet[13]	110	1.7M	6.37	-
	164	1.7M	5.46	24.33
	1001	10.2M	4.92(4.64)	22.71
WRN (ours)	40-4	8.9M	4.53	21.18
	16-8	11.0M	4.27	20.43
	28-10	36.5M	4.00	19.25

Table 5: Test error of different methods on CIFAR-10 and CIFAR-100 with moderate data augmentation (flip/translation) and mean/std normalization. We don’t use dropout for these results. In the second column k is a widening factor. Results for [11] are shown with mini-batch size 128 (as ours), and 64 in parenthesis. Our results were obtained by computing median over 5 runs.

In general, we observed that CIFAR mean/std preprocessing allows training wider and deeper networks with better accuracy, and achieved 18.3% on CIFAR-100 using WRN-40-10 with 56×10^6 parameters (table 9), giving a total improvement of 4.4% over ResNet-1001

Figure 2: Training curves for thin and wide residual networks on CIFAR-10 and CIFAR-100. Solid lines denote test error (y-axis on the right), dashed lines denote training loss (y-axis on the left).

and establishing a new state-of-the-art result on this dataset.

To summarize:

- widening consistently improves performance across residual networks of different depth;
- increasing both depth and width helps until the number of parameters becomes too high and stronger regularization is needed;
- there doesn't seem to be a regularization effect from very high depth in residual networks as wide networks with the same number of parameters as thin ones can learn same or better representations. Furthermore, wide networks can successfully learn with a 2 or more times larger number of parameters than thin ones, which would require doubling the depth of thin networks, making them infeasibly expensive to train.

Dropout in residual blocks

We trained networks with dropout inserted into residual block between convolutions on all datasets. We used cross-validation to determine dropout probability values, 0.3 on CIFAR and 0.4 on SVHN. Also, we didn't have to increase number of training epochs compared to baseline networks without dropout.

Dropout decreases test error on CIFAR-10 and CIFAR-100 by 0.11% and 0.4% correspondingly (over median of 5 runs and mean/std preprocessing) with WRN-28-10, and gives improvements with other ResNets as well (table 6). To our knowledge, that was the first result to approach 20% error on CIFAR-100, even outperforming methods with heavy data augmentation. There is only a slight drop in accuracy with WRN-16-4 on CIFAR-10 which we speculate is due to the relatively small number of parameters.

We notice a disturbing effect in residual network training after the first learning rate drop when both loss and validation error suddenly start to go up and oscillate on high values until the next learning rate drop. We found out that it is caused by weight decay, however making it lower leads to a significant drop in accuracy. Interestingly, dropout partially removes this effect in most cases, see figures 2, 3.

The effect of dropout becomes more evident on SVHN. This is probably due to the fact that we don't do any data augmentation and batch normalization overfits, so dropout adds

depth	k	dropout	CIFAR-10	CIFAR-100	SVHN
16	4		5.02	24.03	1.85
16	4	✓	5.24	23.91	1.64
28	10		4.00	19.25	-
28	10	✓	3.89	18.85	-
52	1		6.43	29.89	2.08
52	1	✓	6.28	29.78	1.70

Table 6: Effect of dropout in residual block. (mean/std preprocessing, CIFAR numbers are based on median of 5 runs)

Figure 3: Training curves for SVHN. On the left: thin and wide networks, on the right: effect of dropout. Solid lines denote test error (y-axis on the right), dashed lines denote training loss (y-axis on the left).

a regularization effect. Evidence for this can be found on training curves in figure 3 where the loss without dropout drops to very low values. The results are presented in table 6. We observe significant improvements from using dropout on both thin and wide networks. Thin 50-layer deep network even outperforms thin 152-layer deep network with stochastic depth [4]. We additionally trained WRN-16-8 with dropout on SVHN (table 9), which achieves 1.54% on SVHN - the best published result to our knowledge. Without dropout it achieves 1.81%.

Overall, despite the arguments of combining with batch normalization, dropout shows itself as an effective technique of regularization of thin and wide networks. It can be used to further improve results from widening, while also being complementary to it.

ImageNet and COCO experiments

For ImageNet we first experiment with non-bottleneck ResNet-18 and ResNet-34, trying to gradually increase their width from 1.0 to 3.0. The results are shown in table 7. Increasing width gradually increases accuracy of both networks, and networks with a comparable number of parameters achieve similar results, despite having different depth. Although these networks have a large number of parameters, they are outperformed by bottleneck networks, which is probably either due to that bottleneck architecture is simply better suited for ImageNet classification task, or due to that this more complex task needs a deeper network. To test this, we took the ResNet-50, and tried to make it wider by increasing inner 3×3 layer width. With widening factor of 2.0 the resulting WRN-50-2-bottleneck outperforms ResNet-152 having 3 times less layers, and being significantly faster. WRN-50-2-bottleneck is only

slightly worse and almost $2\times$ faster than the best-performing pre-activation ResNet-200, although having slightly more parameters (table 8). In general, we find that, unlike CIFAR, ImageNet networks need more width at the same depth to achieve the same accuracy. It is however clear that it is unnecessary to have residual networks with more than 50 layers due to computational reasons.

We didn't try to train bigger bottleneck networks as 8-GPU machines are needed for that.

width		1.0	1.5	2.0	3.0
WRN-18	top1,top5	30.4, 10.93	27.06, 9.0	25.58, 8.06	24.06, 7.33
	#parameters	11.7M	25.9M	45.6M	101.8M
WRN-34	top1,top5	26.77, 8.67	24.5, 7.58	23.39, 7.00	
	#parameters	21.8M	48.6M	86.0M	

Table 7: ILSVRC-2012 validation error (single crop) of non-bottleneck ResNets for various widening factors. Networks with a comparable number of parameters achieve similar accuracy, despite having 2 times less layers.

Model	top-1 err, %	top-5 err, %	#params	time/batch 16
ResNet-50	24.01	7.02	25.6M	49
ResNet-101	22.44	6.21	44.5M	82
ResNet-152	22.16	6.16	60.2M	115
WRN-50-2-bottleneck	21.9	6.03	68.9M	93
pre-ResNet-200	21.66	5.79	64.7M	154

Table 8: ILSVRC-2012 validation error (single crop) of bottleneck ResNets. Faster WRN-50-2-bottleneck outperforms ResNet-152 having 3 times less layers, and stands close to pre-ResNet-200.

We also used WRN-34-2 to participate in COCO 2016 object detection challenge, using a combination of MultiPathNet [62] and LocNet [4]. Despite having only 34 layers, this model achieves state-of-the-art single model performance, outperforming even ResNet-152 and Inception-v4-based models.

Finally, in table 9 we summarize our best WRN results over various commonly used datasets.

Dataset	model	dropout	test perf.
CIFAR-10	WRN-40-10	✓	3.8%
CIFAR-100	WRN-40-10	✓	18.3%
SVHN	WRN-16-8	✓	1.54%
ImageNet (single crop)	WRN-50-2-bottleneck		21.9% top-1, 5.79% top-5
COCO test-std	WRN-34-2		35.2 mAP

Table 9: Best WRN performance over various datasets, single run results. COCO model is based on WRN-34-2 (wider basicblock), uses VGG-16-based AttractionNet proposals, and has a LocNet-style localization part. To our knowledge, these are the best published results for CIFAR-10, CIFAR-100, SVHN, and COCO (using non-ensemble models).

Computational efficiency

Thin and deep residual networks with small kernels are against the nature of GPU computations because of their sequential structure. Increasing width helps effectively balance

computations in much more optimal way, so that wide networks are many times more efficient than thin ones as our benchmarks show. We use cudnn v5 and Titan X to measure forward+backward update times with minibatch size 32 for several networks, the results are in the figure 4. We show that our best CIFAR wide WRN-28-10 is 1.6 times faster than thin ResNet-1001. Furthermore, wide WRN-40-4, which has approximately the same accuracy as ResNet-1001, is 8 times faster.

Figure 4: Time of forward+backward update per minibatch of size 32 for wide and thin networks(x-axis denotes network depth and widening factor). Numbers beside bars indicate test error on CIFAR-10, on top - time (ms). Test time is a proportional fraction of these benchmarks. Note, for instance, that wide WRN-40-4 is 8 times faster than thin ResNet-1001 while having approximately the same accuracy.

Implementation details

In all our experiments we use SGD with Nesterov momentum and cross-entropy loss. The initial learning rate is set to 0.1, weight decay to 0.0005, dampening to 0, momentum to 0.9 and minibatch size to 128. On CIFAR learning rate dropped by 0.2 at 60, 120 and 160 epochs and we train for total 200 epochs. On SVHN initial learning rate is set to 0.01 and we drop it at 80 and 120 epochs by 0.1, training for total 160 epochs. Our implementation is based on Torch [10]. We use [11] to reduce memory footprints of all our networks. For ImageNet experiments we used `fb.resnet.torch` implementation [12]. Our code and models are available at <https://github.com/szagoruyko/wide-residual-networks>.

4 Conclusions

We presented a study on the width of residual networks as well as on the use of dropout in residual architectures. Based on this study, we proposed a wide residual network architecture that provides state-of-the-art results on several commonly used benchmark datasets (including CIFAR-10, CIFAR-100, SVHN and COCO), as well as significant improvements on ImageNet. We demonstrate that wide networks with only 16 layers can significantly outperform 1000-layer deep networks on CIFAR, as well as that 50-layer outperform 152-layer on ImageNet, thus showing that the main power of residual networks is in residual blocks, and not in extreme depth as claimed earlier. Also, wide residual networks are several times faster to train. We think that these intriguing findings will help further advances in research in deep neural networks.

5 Acknowledgements

We thank startup company VisionLabs and Eugenio Culurciello for giving us access to their clusters, without them ImageNet experiments wouldn't be possible. We also thank Adam Lerer and Sam Gross for helpful discussions. Work supported by EC project FP7-ICT-611145 ROBOSPECT.

References

- [1] Yoshua Bengio and Xavier Glorot. Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of AISTATS 2010*, volume 9, pages 249–256, May 2010.
- [2] Yoshua Bengio and Yann LeCun. Scaling learning algorithms towards AI. In Léon Bottou, Olivier Chapelle, D. DeCoste, and J. Weston, editors, *Large Scale Kernel Machines*. MIT Press, 2007.
- [3] Monica Bianchini and Franco Scarselli. On the complexity of shallow and deep neural network classifiers. In *22th European Symposium on Artificial Neural Networks, ESANN 2014, Bruges, Belgium, April 23-25, 2014*, 2014.
- [4] T. Chen, I. Goodfellow, and J. Shlens. Net2net: Accelerating learning via knowledge transfer. In *International Conference on Learning Representation*, 2016.
- [5] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). *CoRR*, abs/1511.07289, 2015.
- [6] R. Collobert, K. Kavukcuoglu, and C. Farabet. Torch7: A matlab-like environment for machine learning. In *BigLearn, NIPS Workshop*, 2011.
- [7] Spyros Gidaris and Nikos Komodakis. Locnet: Improving localization accuracy for object detection. In *Computer Vision and Pattern Recognition (CVPR), 2016 IEEE Conference on*, 2016.
- [8] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning (ICML'13)*, pages 1319–1327, 2013.
- [9] Benjamin Graham. Fractional max-pooling. *arXiv:1412.6071*, 2014.
- [10] Sam Gross and Michael Wilber. Training and investigating residual nets, 2016. URL <https://github.com/facebook/fb.resnet.torch>.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *CoRR*, abs/1502.01852, 2015.

- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. *CoRR*, abs/1603.05027, 2016.
- [14] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. *CoRR*, abs/1603.09382, 2016.
- [15] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In David Blei and Francis Bach, editors, *Proceedings of the 32nd International Conference on Machine Learning (ICML-15)*, pages 448–456. JMLR Workshop and Conference Proceedings, 2015.
- [16] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [17] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-10 (canadian institute for advanced research). 2012. URL <http://www.cs.toronto.edu/~kriz/cifar.html>.
- [18] Hugo Larochelle, Dumitru Erhan, Aaron Courville, James Bergstra, and Yoshua Bengio. An empirical evaluation of deep architectures on problems with many factors of variation. In Zoubin Ghahramani, editor, *Proceedings of the 24th International Conference on Machine Learning (ICML’07)*, pages 473–480. ACM, 2007.
- [19] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-Supervised Nets. 2014.
- [20] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *CoRR*, abs/1312.4400, 2013.
- [21] Francisco Massa. Optnet - reducing memory usage in torch neural networks, 2016. URL <https://github.com/fmassa/optimize-net>.
- [22] Guido F. Montúfar, Razvan Pascanu, KyungHyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada*, pages 2924–2932, 2014.
- [23] Tapani Raiko, Harri Valpola, and Yann Lecun. Deep learning made easier by linear transformations in perceptrons. In Neil D. Lawrence and Mark A. Girolami, editors, *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics (AISTATS-12)*, volume 22, pages 924–932, 2012.
- [24] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. Technical Report Arxiv report 1412.6550, arXiv, 2014.
- [25] J. Schmidhuber. Learning complex, extended sequences using the principle of history compression. *Neural Computation*, 4(2):234–242, 1992.
- [26] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [27] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 2014.

- [28] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *CoRR*, abs/1505.00387, 2015.
- [29] Ilya Sutskever, James Martens, George E. Dahl, and Geoffrey E. Hinton. On the importance of initialization and momentum in deep learning. In Sanjoy Dasgupta and David Mcallester, editors, *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, volume 28, pages 1139–1147. JMLR Workshop and Conference Proceedings, May 2013.
- [30] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [31] Christian Szegedy, Sergey Ioffe, and Vincent Vanhoucke. Inception-v4, inception-resnet and the impact of residual connections on learning. abs/1602.07261, 2016.
- [32] S. Zagoruyko, A. Lerer, T.-Y. Lin, P. O. Pinheiro, S. Gross, S. Chintala, and P. Dollár. A multipath network for object detection. In *BMVC*, 2016.